When using .preload() the WHERE clause conditions for the second table being joined cannot be modified. The resulting SQL query uses an IN clause for the top level query with a list of primary keys from the second table.

If .includes() generates an equivalent SQL query then what value does it add?

With .includes(), Active Record can pick between two strategies. .includes() will work equivalently to .preload() or it will generate a LEFT OUTER JOIN to join the tables together. How does it choose which one to use?

Active Record chooses a LEFT OUTER JOIN when there are conditions on the associated table. What do conditions on the associated table look like?

Modify the example from earlier that fetches Vehicle Reservations, but now fetch the active and not-canceled reservations.

Run the following code in bin/rails console and take a look at the generated SQL query.

ruby/n_plus_one_detection.rb

```
Prosopite.rails_logger = true

Prosopite.scan do
  Vehicle.all.each do |vehicle|
    vehicle.vehicle_reservations.count
  end
end
```

Notice that it's used a LEFT OUTER JOIN in order to accommodate the query conditions for the vehicle reservations.

About about traditional inner joins?

Active Record supports the .joins() method and when a traditional inner join is possible, this will likely be the most efficient type of join.

You've now seen a couple of ways to avoid N+1 queries by using Eager Loading.

What if you could configure your application to make it impossible to write N+1 queries altogether?

Prefer Strict Loading Over Lazy Loading

Active Record introduced a new feature called *Strict Loading* in Rails 6.1[8] that can be used to prevent lazy loading.

Strict Loading can be specified on a per query basis, or globally for an application.

8. https://github.com/rails/rails/pull/37400